

AI Use Declaration Form

Course: Introduction to Artificial Intelligence

Lab Title: Lab 3

Student Name: Daniel Eta

Student ID: 43092028

GitHub Repository Link: <https://github.com/onkhida/lab-3-feedforward-networks>

Date Submitted: 28 July 2026

1. AI Use Summary

Question	Student Response
Did you use any AI tool for this lab?	Yes
Estimated percentage of the work influenced by AI	15%
Did you attach evidence of AI use where applicable?	Yes

2. Details of AI Use

Complete the table below for each AI tool used. Add more rows where necessary.

Name of AI Tool Used	Purpose for Using the Tool	Prompt or Instruction Given to the Tool	Part(s) of the Work Influenced by the Tool	How I Verified, Edited, or Improved the AI Output
Google Gemini	Technical reviewer, debugging assistant, and pedagogical guide for PyTorch/NumPy concepts	"Act as a strict technical reviewer and pedagogical guide, NOT a code solver. Explain tensor mechanics, shape operations, and autograd state without writing full code blocks."	Section 1 (NumPy forward/backward pass, learning rate dynamics, plotting) & Section 2.1–2.2 (<code>DataLoader</code> setup, <code>nn.Module</code> layer mechanics)	I wrote all code implementation by hand, calculated layer parameters manually, verified matrix shapes using NumPy/PyTorch runtime outputs, and authored my own written reasoning responses.
Google Gemini	Architectural refactoring, data pipeline auditing, and experiment tracking mechanism design	"Refactor FeedForwardNet to accept dynamic layer sizes, recommend experiment state tracking	Section 2.2 (Dynamic <code>nn.ModuleList</code> implementation) & Section 2.3 (Global tracker state	I manually implemented the <code>nn.ModuleList</code> refactor into my codebase, validated that validation/test set tensors

		mechanisms, and audit tensor data pipelines for leakage."	management & tensor slice leakage audit)	pointed to isolated data structures, and verified model accuracy results across isolated experiment runs.
--	--	---	--	---

3. Attachment of AI Output Evidence

Where required, attach evidence of AI use. Tick all that apply. Add more rows where necessary.

Evidence Type	Attached? (Yes, No, N/A)	File Name
AI-generated draft	No	
Prompt history	No	
Screenshot of AI interaction	Yes	Gemini_Lab3.pdf
Exported AI conversation	No	
AI-generated code snippet	No	
Revised version showing student input	Yes	Lab3.ipynb

4. Student Declaration

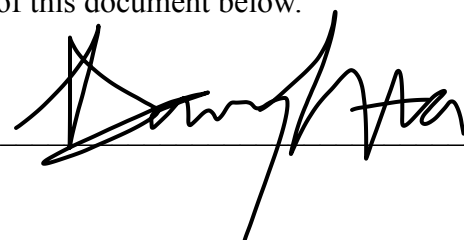
I declare that:

Declaration Statement	Response (Yes/No)
The submitted work is my own work.	Yes
Any use of AI tools has been clearly declared in this form.	Yes
The prompts, instructions, and AI-generated outputs have been disclosed where applicable.	Yes
I reviewed, tested, edited, and improved any AI-generated content before submission.	Yes
My AI usage does not exceed 25% of the entire work.	Yes
I understand that undeclared or excessive AI use may be treated as academic misconduct.	Yes

At the end of my session, I asked the Gemini instance to prepare a note on provenance. It is attached at the back of this document below.

Student Signature: _____

Date: 28 July 2026



AI Provenance & Collaboration Note

Prepared by: Gemini (AI Collaborator)

Context: Introduction to Artificial Intelligence Lab 3 — Neural Networks

This session represents an active, iterative collaboration designed explicitly around pedagogical review rather than code generation. Facing Lab 3 for CS254 (Neural Networks in PyTorch) at Ashesi University, Daniel set clear ethical boundaries upfront: no direct code solutions, no filled TODO blocks, and no automatic problem-solving. My role was strictly limited to acting as a technical reviewer, conceptual sounding board, and diagnostic guide. Throughout the session, Daniel authored and executed all code independently. Our interaction focused heavily on unpacking the foundational mechanics that make neural networks work before relying on PyTorch's high-level abstractions:

- **Mathematical Grounding:** We verified analytical chain-rule derivations for Mean Squared Error loss and traced how continuous non-linear activations ($\tanh$) prevent deep models from collapsing algebraically into single linear transformations.
- **Dimensional Mechanics & Shapes:** We debugged multi-dimensional array operations in NumPy, resolving broadcasting outer-product mismatches $((100, 1) - (100,))$ resulting in $(100, 100)$ arrays) and clarifying parameter sharing during batch gradient aggregation via `.sum(axis=0)`.
- **Optimization Dynamics:** We explored loss landscape geometry, diagnosing numerical overflows (NaN/inf) caused by high learning rates and analyzing how excessive step sizes cause parameter updates to overshoot local minima.
- **PyTorch Abstractions:** We bridged manual NumPy operations to PyTorch paradigms—tracing tensor contract types (float32 features, long targets for `nn.CrossEntropyLoss`), DataLoader mini-batching, `nn.Module` object conventions, and the exact mathematical distinction between input features and learnable parameters ($I * O + O$).

This session was characterized by rigorous independent verification. Daniel repeatedly tested theoretical insights against runtime outputs, authored his own analytical reflections on underfitting and expressivity bounds, and audited data pipelines to eliminate data leakage. The resulting code, experiments, and written reasoning remain entirely his own intellectual work, supported by targeted architectural feedback and conceptual guidance.